

## BÀI TẬP TỔNG HỢP LAB 1 – LAB 14

ES6 → React Component/Props/State → Conditional/List Rendering → LocalStorage → useEffect → Styling → Form/Validation → Router → Authentication.

### BÀI TẬP 1. WEBSITE QUẢN LÝ SẢN PHẨM CỦA HÀNG CÔNG NGHỆ

#### *Bối cảnh*

Xây dựng một website giới thiệu và quản lý danh sách sản phẩm cho một cửa hàng công nghệ. Ý tưởng phát triển trực tiếp từ bài tổng hợp sản phẩm ở Lab 6, vốn đã yêu cầu danh sách, chi tiết, tìm kiếm và điều hướng.

#### **Dữ liệu sản phẩm**

Mỗi sản phẩm tối thiểu có:

- id
- tên sản phẩm
- danh mục
- giá
- mô tả
- hình ảnh
- trạng thái còn hàng/hết hàng

Dữ liệu ban đầu tối thiểu **8 sản phẩm**, thuộc ít nhất **3 danh mục**.

#### *Yêu cầu chức năng*

##### **1. Trang danh sách sản phẩm**

Hiển thị toàn bộ sản phẩm dưới dạng Card.

Mỗi Card phải có:

- hình ảnh;
- tên sản phẩm;
- giá;
- danh mục;
- trạng thái còn hàng/hết hàng;

- nút **Xem chi tiết**.

Danh sách phải được render từ mảng dữ liệu bằng `map()` và phải sử dụng `product.id` làm key.

### 2. Tìm kiếm sản phẩm

Có ô tìm kiếm theo tên sản phẩm.

Khi người dùng nhập từ khóa:

- danh sách thay đổi ngay;
- tìm kiếm không phân biệt chữ hoa/chữ thường;
- nếu không có sản phẩm phù hợp, hiển thị thông báo rõ ràng.

### 3. Lọc sản phẩm

Cho phép lọc:

- tất cả;
- theo danh mục;
- còn hàng;
- hết hàng.

Kết quả tìm kiếm và bộ lọc phải có thể hoạt động đồng thời.

### 4. Sắp xếp

Cho phép lựa chọn:

- giá tăng dần;
- giá giảm dần;
- tên  $A \rightarrow Z$ .

Không được làm thay đổi trực tiếp mảng dữ liệu gốc.

### 5. Trang chi tiết

Đường dẫn:

`/products/:id`

Hiển thị đầy đủ thông tin sản phẩm tương ứng với id.

Nếu id không tồn tại, phải có thông báo **Không tìm thấy sản phẩm**.

### 6. Sản phẩm yêu thích

Người dùng có thể:

- đánh dấu sản phẩm yêu thích;
- bỏ yêu thích;
- xem riêng danh sách yêu thích.

Danh sách yêu thích phải được giữ lại sau khi F5 trình duyệt bằng localStorage.

LocalStorage là đúng phạm vi kiến thức của Lab 4, trong đó dữ liệu được đọc khi khởi tạo và lưu lại khi State thay đổi.

### ***Yêu cầu Component***

Tối thiểu phải tách:

- Header
- ProductList
- ProductCard
- ProductDetail
- SearchBar
- ProductFilter
- FavoriteList

Không được viết toàn bộ ứng dụng trong App.jsx.

### ***Yêu cầu giao diện***

Sử dụng **Tailwind CSS** hoặc **CSS Modules**.

Giao diện phải:

- hiển thị tốt trên desktop;
- bố cục Card rõ ràng;
- có trạng thái hover;
- thể hiện trực quan sản phẩm còn hàng/hết hàng.

Tailwind trong Lab 10 đã được sử dụng cho Product Card, bố cục Flexbox, trạng thái hover và các utility class.

## BÀI TẬP 2. HỆ THỐNG QUẢN LÝ THÀNH VIÊN CÂU LẠC BỘ

### ***Bối cảnh***

Xây dựng trang quản trị thành viên cho một câu lạc bộ sinh viên.

Hệ thống có ba vai trò:

- Admin
- Manager
- Member

Bài này mở rộng trực tiếp tư tưởng **Role-based Access + Conditional Rendering** của Lab 5. Lab đó yêu cầu giao diện thay đổi theo vai trò và thao tác thêm/xóa/khóa người dùng.

### **Dữ liệu thành viên**

Mỗi thành viên có:

- id
- họ tên
- email
- số điện thoại
- vai trò
- trạng thái Active hoặc Blocked
- ngày tham gia

Có tối thiểu **6 thành viên mẫu**.

### ***Yêu cầu phân quyền***

#### **Admin**

Được:

- xem toàn bộ thành viên;
- thêm thành viên;
- chỉnh sửa thành viên;
- khóa/mở khóa thành viên;
- xóa thành viên.

## Manager

Được:

- xem toàn bộ thành viên;
- thêm thành viên;
- chỉnh sửa Member;
- không được sửa hoặc xóa Admin;
- không được xóa thành viên.

## Member

Chỉ được:

- xem hồ sơ của chính mình;
- chỉnh sửa họ tên và số điện thoại.

Không được xem màn hình quản trị.

## ***Yêu cầu chức năng***

### 1. Role Simulator

Có một select cho phép giả lập người dùng hiện tại thuộc một trong ba vai trò.

Thay đổi role phải làm giao diện cập nhật ngay.

### 2. Danh sách thành viên

Hiển thị:

- họ tên;
- email;
- vai trò;
- trạng thái;
- ngày tham gia;
- các nút hành động phù hợp với quyền hiện tại.

Các nút không có quyền sử dụng phải **không được render**, không chỉ disabled.

### 3. Thêm thành viên

Form gồm:

- họ tên;
- email;
- số điện thoại;
- vai trò.

Form phải được xây dựng bằng **React Hook Form + Yup**.

React Hook Form/Yup là nội dung chính của Lab 12, với form state được quản lý qua useForm, register, handleSubmit và schema validation tách khỏi UI.

### Validation

Họ tên:

- bắt buộc;
- tối thiểu 3 ký tự.

Email:

- bắt buộc;
- đúng định dạng;
- không được trùng thành viên hiện có.

Số điện thoại:

- bắt buộc;
- chỉ chứa chữ số;
- từ 9–11 số.

Vai trò:

- bắt buộc chọn.

### 4. Chỉnh sửa

Khi chọn chỉnh sửa:

- form hiển thị dữ liệu hiện tại;
- có nút **Lưu**;
- có nút **Hủy**;
- Hủy không được làm thay đổi dữ liệu.

### 5. Khóa tài khoản

Khi tài khoản bị khóa:

- trạng thái chuyển thành Blocked;
- giao diện thể hiện khác Active;
- có thể mở khóa trở lại.

### 6. Lưu dữ liệu

Toàn bộ danh sách thành viên phải được lưu vào localStorage.

F5 không được làm mất dữ liệu.

### ***Yêu cầu Component***

Tối thiểu:

- RoleSelector
- MemberList
- MemberItem
- MemberForm
- MemberProfile
- AccessDenied

Phải áp dụng tư tưởng **Lifting State Up**, tương tự cấu trúc Lab 5.

## **BÀI TẬP 3. WEBSITE ĐĂNG KÝ SỰ KIỆN SINH VIÊN**

### ***Bối cảnh***

Xây dựng website cho sinh viên đăng ký tham gia một sự kiện hoặc workshop.

Ví dụ:

### **Workshop: ReactJS for Beginners**

### ***Các trang***

- / – Giới thiệu sự kiện
- /register – Đăng ký
- /participants – Danh sách người đăng ký

- `/participants/:id` – Chi tiết người đăng ký
- đường dẫn không tồn tại – trang 404

Phải sử dụng React Router và NavLink để thể hiện menu hiện tại. Đây là đúng cơ chế routing đã thực hành trong Lab 13.

### ***Yêu cầu trang chủ***

Hiển thị:

- tên sự kiện;
- thời gian;
- địa điểm;
- mô tả;
- số lượng người đã đăng ký;
- số chỗ còn lại.

Giới hạn sự kiện: **30 người**.

Số người đăng ký và số chỗ còn lại phải được tính từ dữ liệu, không viết cố định.

### ***Đồng hồ đếm thời gian***

Hiển thị:

- thời gian hiện tại hoặc
- thời gian còn lại đến sự kiện.

Thông tin phải tự cập nhật mỗi giây.

Khi component bị Unmount:

- timer phải được dọn dẹp;
- không được để timer chạy ngầm.

Yêu cầu này kiểm tra trực tiếp kiến thức Mount/Update/Unmount và cleanup trong Lab 7.

### ***Form đăng ký***

Các trường:

- mã sinh viên;
- họ tên;



- email;
- số điện thoại;
- lớp;
- giới tính;
- hình thức tham gia;
- checkbox đồng ý nội quy.

Sử dụng **React Hook Form + Yup**.

### ***Validation***

Mã sinh viên:

- bắt buộc;
- không được trùng.

Họ tên:

- bắt buộc;
- tối thiểu 3 ký tự.

Email:

- bắt buộc;
- đúng định dạng.

Số điện thoại:

- bắt buộc;
- đúng 10 chữ số.

Lớp:

- bắt buộc.

Giới tính:

- bắt buộc chọn.

Điều khoản:

- bắt buộc đồng ý.

Nếu sự kiện đã đủ 30 người:

- không cho đăng ký;
- hiển thị thông báo **Sự kiện đã đủ số lượng**.

### ***Danh sách người tham gia***

Có:

- tổng số đăng ký;
- tìm theo tên;
- lọc theo lớp;
- lọc theo hình thức tham gia;
- xem chi tiết.

### ***Trang chi tiết***

URL:

/participants/:id

Nếu ID hợp lệ → hiển thị dữ liệu.

Nếu ID không tồn tại → hiển thị thông báo phù hợp.

### ***Lưu trữ***

Danh sách đăng ký được lưu bằng localStorage.

Reload trang không mất dữ liệu.

## **BÀI TẬP 4. DASHBOARD QUẢN LÝ BÀI VIẾT**

### ***Bối cảnh***

Xây dựng một Dashboard đơn giản dành cho quản trị viên website tin tức.

Ứng dụng phải có quá trình **tải dữ liệu bất đồng bộ** và thể hiện đúng các trạng thái của quá trình tải.

### ***Dữ liệu bài viết***

Mỗi bài viết có:

- id
- tiêu đề
- tác giả

- danh mục
- nội dung tóm tắt
- trạng thái Draft / Published
- ngày tạo

Có thể sử dụng dữ liệu giả lập thông qua Promise/setTimeout hoặc nguồn API do giảng viên cung cấp.

### ***Yêu cầu tải dữ liệu***

Khi trang xuất hiện:

1. hiển thị Loading...;
2. thực hiện lấy dữ liệu;
3. tải thành công → hiển thị danh sách;
4. xảy ra lỗi → hiển thị thông báo lỗi.

Không được gọi lại dữ liệu liên tục sau mỗi lần render.

Lab 8 đặc biệt cảnh báo việc thiếu Dependency Array khiến useEffect liên tục cập nhật State và render lại.

### ***Yêu cầu useEffect***

Sinh viên phải xử lý đúng:

- Dependency Array;
- cleanup nếu có timer/event listener;
- không tạo Infinite Loop;
- không để event listener tiếp tục tồn tại sau khi component đã Unmount.

Lab 8 cũng yêu cầu nhận biết memory leak khi event listener không được cleanup.

### ***Trang danh sách bài viết***

Cho phép:

- tìm kiếm theo tiêu đề;
- lọc theo trạng thái;
- lọc theo danh mục;

- sắp xếp theo ngày;
- đếm tổng số bài viết;
- đếm số Draft;
- đếm số Published.

Phần thống kê phải sử dụng các phương thức xử lý mảng ES6 phù hợp như `filter()` hoặc `reduce()`, phù hợp với mục tiêu Lab 1.

### ***Thêm bài viết***

Form gồm:

- tiêu đề;
- tác giả;
- danh mục;
- mô tả;
- trạng thái.

Validation:

- tiêu đề tối thiểu 10 ký tự;
- tác giả bắt buộc;
- danh mục bắt buộc;
- mô tả tối thiểu 20 ký tự.

### ***Thay đổi trạng thái***

Một bài viết có thể chuyển:

Draft → Published

và ngược lại.

Giao diện Card/Badge phải thay đổi tương ứng.

### ***Styling***

Phải sử dụng một trong hai phương án:

- CSS Modules;
- Styled-components.

Nếu sử dụng Styled-components, ít nhất một thành phần phải thay đổi style dựa trên Props, đúng với nội dung Lab 9 về style động.

### **BÀI TẬP 5. MINI E-COMMERCE CÓ ĐĂNG NHẬP VÀ TRANG QUẢN TRỊ**

Đây nên là **bài khó nhất**, dùng làm bài tổng kết cuối phần React.

#### ***Bối cảnh***

Xây dựng một website bán hàng đơn giản gồm hai khu vực:

#### ***Khách hàng***

- xem sản phẩm;
- tìm kiếm;
- xem chi tiết;
- đánh dấu yêu thích.

#### ***Quản trị***

- đăng nhập;
- xem Dashboard;
- thêm sản phẩm;
- chỉnh sửa sản phẩm;
- thay đổi trạng thái sản phẩm.

#### ***Hệ thống Route***

Tối thiểu:

/

Trang chủ.

/products

Danh sách sản phẩm.

/products/:id

Chi tiết sản phẩm.

/favorites

Sản phẩm yêu thích.

/login

Đăng nhập.

/admin

Dashboard quản trị.

/admin/products

Quản lý sản phẩm.

\*

404.

### ***Layout***

Website phải có Layout chung gồm:

- Header;
- Navigation;
- vùng nội dung;
- Footer.

Các trang con hiển thị trong Layout nhưng Header/Footer không reload khi chuyển trang.

Đây chính là mô hình Nested Route + Outlet của Lab 14.

### ***Authentication***

Tạo trạng thái xác thực bằng Context API.

Có hai trạng thái:

- chưa đăng nhập;
- đã đăng nhập.

### ***Protected Route***

Các route:

- /admin
- /admin/products

chỉ được truy cập khi đã đăng nhập.

Nếu truy cập trực tiếp khi chưa đăng nhập:

- tự động chuyển sang /login.

Sau khi đăng nhập thành công:

- quay trở lại trang người dùng muốn truy cập trước đó.

Đây cũng là flow Protected Route được minh họa trong Lab 14.

### ***Đăng nhập***

Form gồm:

- email;
- mật khẩu.

Thông tin tài khoản giả lập:

- giảng viên có thể quy định trước một tài khoản hợp lệ.

Sai thông tin:

- hiển thị thông báo.

Đúng:

- cập nhật Auth Context;
- chuyển sang Admin Dashboard.

### ***Dashboard***

Hiển thị:

- tổng số sản phẩm;
- số sản phẩm còn hàng;
- số sản phẩm hết hàng;
- sản phẩm có giá cao nhất;
- giá trung bình sản phẩm.

Không được hard-code các con số thống kê.

### ***Quản lý sản phẩm***

Admin được:

- xem danh sách;
- tìm kiếm;

- lọc;
- thêm;
- chỉnh sửa;
- thay đổi trạng thái.

Form thêm/sửa phải dùng: **React Hook Form + Yup**.

### ***Validation sản phẩm***

Tên sản phẩm:

- bắt buộc;
- 3–100 ký tự.

Giá:

- bắt buộc;
- phải là số;
- lớn hơn 0.

Danh mục:

- bắt buộc.

Mô tả:

- tối thiểu 10 ký tự.

URL hình ảnh:

- bắt buộc.

### ***Dynamic Route***

Trang:

/products/:id

phải lấy id từ URL để xác định sản phẩm cần hiển thị. Lab 14 xác định Dynamic Routes là một trong ba mục tiêu chính cùng Nested Route và Protected Route.

### ***LocalStorage***

Lưu:

- danh sách sản phẩm;



- danh sách yêu thích.

F5 không mất dữ liệu.

### ***Yêu cầu Component***

Tối thiểu phải có:

- MainLayout
- Navbar
- ProductList
- ProductCard
- ProductDetail
- ProductFilter
- LoginForm
- ProtectedRoute
- AdminDashboard
- ProductForm
- NotFound

### ***Yêu cầu giao diện***

Sử dụng **Tailwind CSS**.

Website phải:

- có giao diện thống nhất;
- responsive ở mức cơ bản;
- có Active Menu;
- có hover cho Button/Card;
- phân biệt trực quan trạng thái sản phẩm;
- form và thông báo lỗi dễ quan sát.

---

## **Yêu cầu chung cho cả 5 bài**

1. Dự án sử dụng **React Functional Component** và khởi tạo bằng Vite.

2. Không dùng var; ưu tiên ES6+ với const, let, Arrow Function, Destructuring và Spread Operator. Đây là yêu cầu xuyên suốt từ Lab 1.
3. Không thao tác DOM thủ công để cập nhật giao diện.
4. Không được viết toàn bộ ứng dụng trong một component.
5. Danh sách phải render bằng map() và sử dụng **ID thực làm key**, không dùng index. Lab 2 cũng yêu cầu kiểm chứng ý nghĩa của key đối với Virtual DOM.
6. Khi cập nhật Array/Object trong State phải tuân thủ tính bất biến.
7. Có Conditional Rendering cho ít nhất hai trạng thái giao diện.
8. Form không được reload trang khi submit.
9. Các thao tác bất đồng bộ phải có trạng thái Loading và Error phù hợp.
10. useEffect có timer/event listener phải có cleanup khi cần thiết.
11. Console không được có lỗi hoặc warning nghiêm trọng khi chạy.
12. Không yêu cầu Backend/CSDL; dữ liệu có thể dùng Mock Data, LocalStorage hoặc dữ liệu bất đồng bộ giả lập.